E2: the cofactor ladder, where the abscissa base and a known-log base separate

PLAN §4 E2 (unchanged by Amendment 1): population P1, the same arms as E1, at cofactor `h ∈ {1, 2, 4, 8, 16}`. **No security claim.** The known-log base is a generic precomputation table in the Bernstein–Lange shape, and no exponent moves. **P-192 and P-224 have `h = 1`, so E2 says nothing about them.**

PROTOCOL (frozen before any measured run)

Frozen in the commit that adds this section, together with `e2/curves.jsonl`, `e2/prereg.jsonl` and the 40 specs `e2/specs/h*-c*.json`. No E2 record existed when it was written. Only the "Results" sections below are written after the run.

Population P1

- 24-bit `p`, curves `y² = x³ − 3x + b` (`a = −3`, as NIST), `#E = h·r` with `r` prime and `r > h`, 8 curves per `h`, 40 in all.
- Found by `examples/a_minus_3_prime_order_curves.rs -- 24 8 20261019 H` (seed 20261019 for every `h`). At `h = 1` this is the E1 finder unchanged, and its first two curves are DRYRUN.md's c1 and c2. At `h > 1`, `#E` is the one multiple of a point's order in the Hasse interval, and `G = [h]P` for the smallest-abscissa `P` with `[h]P ≠ ∞`, so `G` has order `r`. Because `r` is prime and `r ∤ h`, `⟨G⟩` is the only subgroup of order `r`, and any other generator gives the same subgroup (§1B).
- `#E` of every curve is re-counted by `PrimeCurve::point_count` (`O(p)` Legendre symbols) in `generator_effect_e2_n0` before anything is predicted: 40 of 40 agree.

Arms (per curve, all at the same `B`)

`B = ⌈#E^{1/3}⌉` (`family_optimum_base(#E, 1, 1)`, the family optimum; 205 to 256 here). Oracle `mitm:negation_folded=1,m=2`, walk targets, for every IC arm.

arm	method	role
`rho-neg`	`rho.negation`	matched rho reference
`ic-abscissa`	`ic.pipeline`, `prime-abscissa:size=B`	baseline
`ic-knownlog-random`	`ic.pipeline`, `known-log:size=B,scalars=random,seed=20261019`	candidate
`ic-knownlog-random-aa`	the same with `seed=20261020`	A/A of the candidate: a second, independent random base of the same law

The A/A uses another scalar seed because `ecbench` gives every arm the same algorithm seed per (workload, round): a duplicate arm with identical parameters would reproduce the candidate exactly and test nothing.

Workloads and measurement

- One `ecbench` session per curve (`e2/specs/h{H}-c{i}.json`, 40 in all; `B` differs per curve and an arm's parameters are fixed per session): **768 `planted_shared` targets per curve**, target seed 20261019, 1 measured round, no warm-up, order `alternate`, seed 20261019, isolation L0, timeout 600 s per run.
- **Sizing.** A known-log run stops at its first relation, so each `h` gives `8 × 768 = 6144` known-log relations. The relative standard error of their pooled hit rate is about `1/√6144 = 1.28%`, and the abscissa runs add little (each collects tens of relations). That gives a 90% half-width of about **±2.1%**

on the yield ratio, inside the ±5% target. It also leaves room to resolve the 3.3% gap between the two predictions at `h = 16` (below).
- One host, shared (L0). Operation counts and GAE are exact and deterministic. Wall time is reported only as run time.

Predictions (computed before the run, `e2/prereg.jsonl`)

`examples/generator_effect_e2_n0.rs` builds each arm's base exactly as the pipeline does and records, per base:

- N_0, the class-cancelling count of `exact_decomposition_ceiling` (`ic_boundary.rs:451`, `m = 2`, over the signed points);
- `sumset`, the number of distinct nonzero pair sums $\pm P_i \pm P_j$ ($i \le j$) lying in $\langle G \rangle$. This is exactly the set the negation-folded pair table can match, and the predicted relations per target tried is `sumset / r`.

N_0 counts multisets; the table holds distinct sums. They differ by the birthday collisions among sums. These are negligible for the abscissa base (only $\approx 1/h$ of its sums lie in $\langle G \rangle$), but they grow with h for a known-log base, whose $\approx 2B^2$ sums all lie in a subgroup of order $r \approx \#E/h$. That subgroup is 12% to 14% full at $h = 16$.

Predicted online-yield ratio known-log / abscissa (per curve, then the mean of the 8):

h	N_0 ratio, range (mean)	sumset ratio, range (mean)	sumset / N_0 (mean)	A/A sumset ratio, range
1	1.0000 – 1.0000 (1.0000)	0.9989 – 1.0011 (0.9999)	0.9999	0.9984 – 1.0006
2	1.9737 – 1.9961 (1.9913)	1.9678 – 1.9918 (1.9863)	0.9975	0.9986 – 1.0010
4	3.8293 – 3.9898 (3.9324)	3.8048 – 3.9625 (3.9076)	0.9937	0.9982 – 1.0039
8	7.5971 – 7.9356 (7.8207)	7.4748 – 7.8320 (7.7097)	0.9858	0.9967 – 1.0016
16	14.824 – 15.528 (15.298)	14.262 – 15.043 (14.795)	0.9671	0.9952 – 1.0034

At $h = 1$ every base has one class, so the N_0 ratio is exactly 1.000 on every curve: this is the control that ties E2 to E1. The ratio falls short of h because the smallest abscissae do not spread evenly over the h classes.

Statistics

`examples/generator_effect_e2_summary.rs`. For each arm, `O/E` is the relations found over Σ `targets_tried × predicted rate`, pooled over the curves of one h. The **measured / predicted yield ratio** is `O/E(known-log) / O/E(abscissa)`; on one curve it is `(y_kl / y_abs) / (pred_kl / pred_abs)`. Intervals are percentile 90% over 10 000 bootstrap resamples (seed 20261019) of targets within each curve. A two-level interval (curves, then targets) is reported beside it as a sensitivity check, with no decision weight. Total S is `(GAE + w · unpriced ops) / √r` at $w \in \{0, 0.1, 1\}$ (AGENTS.md §5a). Ratios are ratios of means over the same targets.

Decision rules

1. **Primary (PLAN E2):** at each h, the N_0 prediction holds if the 90% interval of measured / predicted (N_0) contains 1. Expected from the collision count: it holds at $h \le 4$. At $h = 8$ and $h =$

16 it is expected to read about 0.986 and 0.967, and at h = 16 the interval should exclude 1. That would be the predicted N_0 over-count, not a failure of the class mechanism.

2. **Sumset prediction (exact):** at each h, the 90% interval of measured / predicted (sumset) contains 1. If this fails at any h, the mechanism is reported as incomplete there.
3. **Controls:** h = 1, both predictions contain 1. A/A, at every h, measured / predicted (sumset) of aa / candidate contains 1. A control failure voids the h it fails at.
4. **Width:** the 90% half-width of measured / predicted is at most 5% at every h. If it is wider, the h is reported as underpowered.
5. **Total S (§5a):** known-log over abscissa and both over rho, paired by target. An order is stated only where the interval is on one side of 1 at all three weights. Expected direction, with no decision weight: known-log below abscissa at every w and h. Gap to rho: reported as distance remaining, not as a gate.
6. Every run that fails to verify is listed with its recorded reason.

Change made before freezing (found in the smoke test)

A two-target smoke test on an h = 16 curve showed ic-knownlog-random finding its relation and then ending exhausted. The relation loop's unknowns are the logs of [h]P_v (its rows read Σ c·x − h·b·d = h·a), but the known-log solver was handed s_v, the log of P_v. The pipeline now hands it h·s_v mod r (ic_framework/mod.rs). At h = 1 that is the identity, so E1′s dry-run records are unaffected. A unit test recovers the planted log on this E2 h = 16 curve with three seeds.

The smoke runs (2, then 16 targets, on the first h = 16 curve, after the fix: 64 of 64 records verified) are scratch runs that check the tooling. They are not part of E2′s data and are not used in any number below.

What ran

item	value
commit of the frozen PROTOCOL	`abc53177e` (`E2: freeze PROTOCOL, population and N0/sumset predictions before any run`)
sessions	40 `ecbench` sessions, one per curve, `e2/records/h{H}-c{i}/` (ids in the appendix)
records	122 880 (40 × 768 targets × 4 arms), **122 880 verified**, 0 failed, 0 timed out; `ecbench verify` audit OK on all 40, and `--replay 16` reproduced 16 of 16 runs on `h1-c1` and `h16-c8`
host	one shared macOS host, isolation L0 (no CPU topology, `--cpus none`), five sessions at a time with separate lock files; no wall time is used in any result
run time	22:13 to 22:35 UTC on 2026-10-19 (about 22 min wall, five streams in parallel); 505 s of summed in-method solve time, 26 to 38 s per IC arm per `h` and 1 to 3 s for rho
aggregation	`examples/generator_effect_e2_summary.rs` → `e2/summary.json`
graph	`e2/e2.svg` (PNG beside it), source `e2/plot_e2.py`, which plots recorded values only
run report	`reports/run-20261019T2213Z/`

Result 1: the yield ratio follows the class count (primary)

All values are measured / predicted for the known-log over abscissa online yield ratio, pooled over the 8 curves of each `h` (`O/E`), with 90% intervals from resampling targets within curves. The two-level interval (curves, then targets) is in `summary.json` and is at most 2.5 points wider in total.

h	measured ratio, pooled	predicted N_0 (mean)	**measured / N_0**	predicted sumset (mean)	**measured / sumset**
1	1.008 [0.987, 1.030]	1.0000	**1.002 [0.982, 1.024]**	0.9999	**1.002 [0.982, 1.024]**
2	2.015 [1.972, 2.057]	1.991	**1.008 [0.987, 1.029]**	1.986	**1.011 [0.989, 1.032]**
4	3.997 [3.915, 4.084]	3.932	**1.010 [0.990, 1.032]**	3.908	**1.017 [0.996, 1.039]**
8	7.832 [7.676, 7.991]	7.821	**0.997 [0.977, 1.017]**	7.710	**1.011 [0.991, 1.032]**
16	14.99 [14.70, 15.29]	15.30	**0.975 [0.957, 0.995]**	14.79	**1.008 [0.989, 1.029]**

Against the frozen rules:

1. **N_0 (rule 1).** The prediction holds at `h = 1, 2, 4, 8`. At `h = 16` the interval excludes 1 (0.975 [0.957, 0.995]). The PROTOCOL expected this (about 0.967), because N_0 counts multisets while the

table holds distinct sums. With 12% to 14% of ⟨G⟩ covered by the known-log sums, birthday collisions remove about 3% of them.

2. **Sumset (rule 2).** The interval contains 1 at every h.
3. **Controls (rule 3).** At h = 1, both predictions are exactly 1.000 (one class per base) and measured / predicted is 1.002 [0.982, 1.024]. That ties E2 to E1: DRYRUN.md's c1 and c2 are curves 1 and 2 here, and dry run 2 measured 0.989 and 0.976 on them. **The A/A fails at h = 4**: aa / candidate measured / predicted is 0.971 [0.943, 0.9998], with an upper end 0.0002 below 1. **So h = 4 is void under the frozen rule**, and its row above is descriptive only. For context, not as a rescue: each of the two random bases on its own matches its sumset rate at h = 4 (1.013 [0.993, 1.035] and 0.984 [0.963, 1.005]), and the two-level A/A interval, [0.938, 1.006], contains 1. With five A/A intervals at 90%, about a 41% chance that at least one misses is expected under the null. At h = 1, 2, 8, 16 the A/A reads 1.012, 0.997, 0.985 and 1.005, and every interval contains 1.
4. **Width (rule 4).** The 90% half-width is 2.0 to 2.2 points at every h, inside ±5%.

So on fresh curves, at h ∈ {1, 2, 8, 16} (with h = 4 void), the abscissa base's online yield is lower than a base inside ⟨G⟩ by the ratio of their subgroup sumsets, |2Σ ∩ ⟨G⟩|. That is about h, and 0.95 h to 0.99 h on these curves because the smallest abscissae do not spread evenly over the classes. N_0, the bound E3a used, predicts the ratio to ±1% up to h = 8 and over-predicts by 2.5% at h = 16.

Absolute rates (measured / predicted from the sumset, pooled): known-log 1.001 to 1.013 and every interval contains 1. Abscissa 0.996 to 0.999, with intervals of about ±0.2 points: at h = 2, 4, 8 the interval sits 0.2 to 0.4 points below 1, and at h = 1 and h = 16 it touches or contains 1. This small deficit of the abscissa arm is recorded as measured and is not explained here. It is an order of magnitude below the ratio intervals and does not change any rule above.

Result 2: total S at T = 1, known-log over abscissa, through the §5a band

Mean S per target, 6144 targets per arm per h; known-log over abscissa paired by target.

h	w	rho-neg	abscissa	known-log random	**known-log / abscissa**
1	0	1.11	19.81	17.76	**0.897 [0.896, 0.898]**
1	0.1				0.859 [0.858, 0.860]
1	1	2.11	28.49	17.83	**0.626 [0.625, 0.627]**
2	0	1.11	27.83	25.23	**0.907 [0.906, 0.908]**
2	0.1				0.872 [0.871, 0.873]
2	1	2.08	38.90	25.27	**0.650 [0.648, 0.651]**
4	0	1.16	37.98	35.14	**0.925 [0.924, 0.926]** (void `h`)
4	0.1				0.894 [0.893, 0.895]
4	1	2.23	51.31	35.17	**0.685 [0.684, 0.687]**
8	0	1.22	52.56	49.63	**0.944 [0.943, 0.945]**
8	0.1				0.916 [0.915, 0.918]
8	1	2.31	68.54	49.65	**0.724 [0.723, 0.726]**
16	0	1.32	71.12	68.75	**0.967 [0.966, 0.968]**
16	0.1				0.943 [0.941, 0.944]
16	1	2.45	89.31	68.77	**0.770 [0.768, 0.772]**

- **The order holds across the band at every `h`.** Known-log is below the abscissa baseline at `w = 0`, 0.1 and 1, so under §5a it ranks below, by 3% to 10% at `w = 0` and 23% to 37% at `w = 1`. At `h = 1` this repeats dry run 2′s 0.89 / 0.85 / 0.61 on 8 curves instead of 2.
- **The saving shrinks as `h` grows; it does not grow.** Phases at `w = 0`: the pair-table build is the same in both arms and rises from 15.3 S (`h = 1`) to 60.7 S (`h = 16`). The table is about `B² ≈ #E^{2/3}` additions, measured in units of `√r = √(#E/h)`. The abscissa arm's relations phase grows only from 4.5 to 10.4 S, because it stops when the target is pinned, after tens of relations, not `B`. The known-log arm pays 2.1 to 7.0 S for `B` scalar multiplications. The class loss multiplies the abscissa arm's targets per relation by about `h`, but the table, which both arms build, grows faster.
- **The A/A total S** reads 1.001 [1.001, 1.001] at every `h`. That is the cost of scalar multiplication by the other seed's scalars (base construction, 2.15 against 2.13 S at `h = 1`), not noise.

Result 3: the gap to rho (distance remaining)

Total S over `rho-neg`, paired by target, 90% intervals:

h	known-log, w = 0 / 0.1 / 1	abscissa, w = 0 / 0.1 / 1
1	16.0 [15.9, 16.2] / 14.7 / 8.4 [8.4, 8.5]	17.9 [17.7, 18.0] / 17.1 / 13.5 [13.4, 13.6]
2	22.7 [22.5, 22.9] / 20.9 / 12.2 [12.0, 12.3]	25.0 [24.8, 25.3] / 24.0 / 18.7 [18.5, 18.9]
4	30.2 [29.9, 30.5] / 27.7 / 15.8 [15.6, 15.9]	32.6 [32.3, 33.0] / 31.0 / 23.0 [22.8, 23.3]
8	40.6 [40.2, 41.0] / 37.2 / 21.5 [21.3, 21.7]	43.0 [42.6, 43.4] / 40.6 / 29.6 [29.3, 30.0]
16	51.9 [51.5, 52.4] / 47.9 / 28.0 [27.8, 28.3]	53.7 [53.2, 54.2] / 50.8 / 36.4 [36.0, 36.8]

Both IC arms are 8 to 54 times rho at `T = 1`, and the gap widens with `h` because the table's cost is set by `#E` and the unit by `r`. Dry run 2 already showed that BSGS at the base's size is cheaper than the known-log arm at `h = 1`. E2 did not re-run BSGS.

Reading

- **The cofactor-class mechanism of §1E is confirmed on fresh data.** At equal `B` the online yield of the smallest-abscissa base is the subgroup sumset over `r`, and a base inside `⟨G⟩` beats it by the ratio of subgroup sumsets (about `h`, within ±2.2 points at `h ∈ {1, 2, 8, 16}`). N_0 is the right quantity up to `h = 8` and over-counts by 2.5% at `h = 16`, where the sums collide. This is E3a's explanation, measured directly rather than regressed.
- **Fixing `G` helps only through the table, not through the base.** A base of known multiples of `G` removes the matrix and realises the full subgroup sumset. That makes it cheaper than the abscissa baseline at every `h` and every band weight (3% to 37%). The saving narrows as `h` grows, because the pair table both arms build dominates. It is the generic-precomputation shape (§2), and it stays 8 to 52 times rho here.
- **Scope.** 24-bit `a = −3` curves with cofactor 1 to 16, MITM with `m = 2`, `B = ⌈#E^{1/3}⌉`, `T = 1`. **P-192 and P-224 have `h = 1`, so E2 says nothing about them** beyond the `h = 1` row, which is E1′s setting. No security claim, and no exponent moves.
- **Under the CLAUDE.md criterion**, the known-log row's saving over the abscissa baseline is a measured improvement of the IC baseline inside the family on these toy curves (and §2 says what kind). It is not a change to any canonical leaderboard row: the leaderboard's rows are other curves and configurations, and nothing here was registered there. The curves are not in `docs/curves/registry.json`; register them before citing them in prose outside this study (AGENTS.md §11).

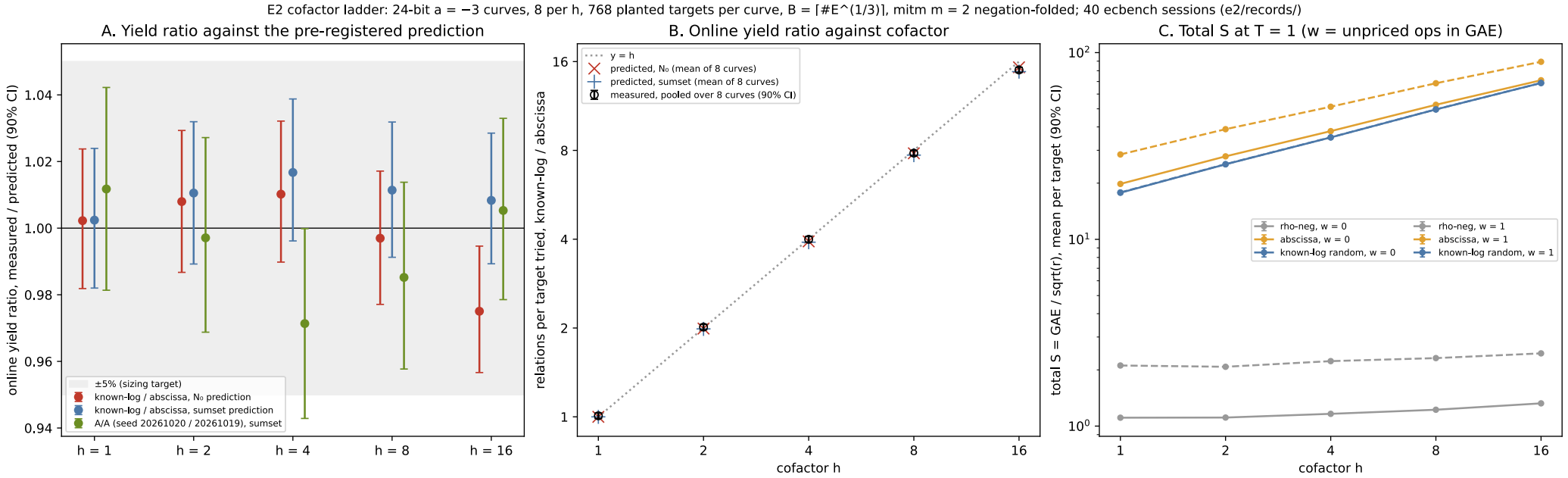


Figure 1: E2 cofactor ladder

Figure 1. A: known-log over abscissa online yield ratio, measured over predicted, with the N_0 prediction (red) and the sumset prediction (blue), and the A/A (green), with 90% intervals and the ±5% sizing band. B: measured yield ratio against h, with both predictions and $y = h$. C: mean total S per target at $w = 0$ (solid) and $w = 1$ (dashed) for rho-neg, abscissa and known-log. 40 ecbench sessions (appendix), 768 targets × 4 arms per curve, 8 curves per h.

Appendix: session ids

h	sessions (curves c1 … c8, `e2/records/h{H}-c{i}`)
1	ECBS1h863af11cdcf0 ECBS1h52db9220ce90 ECBS1hde6946991b0d ECBS1h29896ad26162 ECBS1hbb883d52113b ECBS1h8a36a0907276 ECBS1h230c12cbc5a5 ECBS1hcd730a01c052
2	ECBS1h74db8f039110 ECBS1hcdf5afdcfc4f ECBS1hb1063363545f ECBS1hb17af2ea5f5a ECBS1h92a627fc30fd ECBS1h1b0fc7d2a9e1 ECBS1h8c221127348e ECBS1heb366a6c06f2
4	ECBS1hffff4e65ef85 ECBS1hfa5409e80899 ECBS1ha82b101079bb ECBS1h94f4fe0c0bed ECBS1h9bb674d12547 ECBS1h936064102563 ECBS1h8703d6045f7d ECBS1hdd17ae66b512
8	ECBS1hbb0bf07c0a87 ECBS1hedc7a19e6683 ECBS1h1aef6a9375d2 ECBS1h2f768636b0f8 ECBS1h91e66ac0c74f ECBS1h42761b37eaea ECBS1h3530519a9bca ECBS1hec63397b3b4b
16	ECBS1he89077ecc0d2 ECBS1he2d56ff85cee ECBS1h761a07cb4b75 ECBS1h724149cd1998 ECBS1h9bc6b0e30630 ECBS1h5d80f65ac099 ECBS1h849628c19500 ECBS1h9d620de43cfb

Each session's `records.jsonl` and `plan.json` are stored gzipped (`*.jsonl.gz`, `plan.json.gz`). Run `gunzip` on them before `ecbench verify --dir` or before re-running the summary.